

งานเทศกาล

ในเรากำลังเล่นเกมอยู่ในงานเทศกาลแห่งหนึ่ง ซึ่งมีรางวัลใหญ่เป็นทริปไปลา구나โคโลราดา เกมนี้ประกอบไปด้วยการใช้เหรียญเพื่อซื้อคูปอง โดยการซื้อคูปองใบหนึ่งอาจทำให้มีเหรียญเพิ่มขึ้นได้ เป้าหมายคือการทำให้ได้จำนวนคูปองมากที่สุดเท่าที่เป็นไปได้

เธอเริ่มเล่นเกมโดยมีเหรียญอยู่ A เหรียญ มีคูปองทั้งหมด N ใบ ซึ่งถูกกำกับด้วยหมายเลขตั้งแต่ 0 ถึง $N - 1$ ในเราต้องจ่าย $P[i]$ เหรียญ ($0 \leq i < N$) เพื่อที่จะซื้อคูปองใบที่ i (เธอต้องมีเหรียญอย่างน้อย $P[i]$ เหรียญก่อนการซื้อคูปองใบที่ i) เธอสามารถซื้อคูปองแต่ละใบได้มากที่สุดหนึ่งครั้ง

นอกจากนี้แต่ละคูปองใบที่ i ($0 \leq i < N$) ได้รับการกำหนด **ประเภท** แสดงด้วย $T[i]$ ซึ่งเป็นจำนวนเต็ม **ระหว่าง 1 และ 4 รวมหัวท้าย** หลังจากที่ในเราซื้อคูปองใบที่ i แล้ว จำนวนเหรียญที่เรามีเหลือจะถูกคูณด้วย $T[i]$ กล่าวอย่างเป็นทางการได้ว่า ถ้าในขณะหนึ่งในเกมเรามีเหรียญ X เหรียญ และซื้อคูปองใบที่ i (ซึ่งต้องใช้ $X \geq P[i]$) เธอจะมีเหรียญ $(X - P[i]) \cdot T[i]$ เหรียญหลังการซื้อ

งานของคุณคือการตัดสินใจว่าในเราควรซื้อคูปองใบบ้าง และซื้อในลำดับใด เพื่อทำให้จำนวนคูปองทั้งหมดที่เรามีในตอนท้ายมีค่ามากที่สุด หากมีการสั่งซื้อมากกว่าหนึ่งลำดับที่บรรลุผลดังกล่าว คุณสามารถรายงานลำดับใดก็ได้

รายละเอียดการเขียนโปรแกรม

คุณจะต้องเขียนฟังก์ชันต่อไปนี้

```
std::vector<int> max_coupons(int A, std::vector<int> P,
                             std::vector<int> T)
```

- A : จำนวนเหรียญที่ในเรามีตอนเริ่มต้น
- P : อาร์เรย์ความยาว N ระบุราคาของคูปอง
- T : อาร์เรย์ความยาว N ระบุประเภทของคูปอง
- ฟังก์ชันนี้จะถูกเรียกเพียงหนึ่งครั้งสำหรับแต่ละกรณีทดสอบ

ฟังก์ชันต้องคืนค่าเป็นอาร์เรย์ R ที่ระบุการซื้อของในเราดังต่อไปนี้:

- ความยาวของ R ต้องเป็นจำนวนคูปองที่มากที่สุดที่เธอสามารถซื้อได้
- สมาชิกในอาร์เรย์เป็นหมายเลขของคูปองที่เธอซื้อตามลำดับเวลา นั่นคือ ในเราซื้อคูปองใบที่ $R[0]$ เป็นลำดับแรกตามด้วยคูปองใบที่ $R[1]$ เป็นลำดับที่สอง เช่นนี้ไปเรื่อย ๆ
- สมาชิกแต่ละตัวใน R จะต้องไม่ซ้ำกัน

R ควรเป็นอาร์เรย์ว่างหากไม่สามารถซื้อคูปองได้

เงื่อนไข

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ สำหรับแต่ละ i ที่ $0 \leq i < N$
- $1 \leq T[i] \leq 4$ สำหรับแต่ละ i ที่ $0 \leq i < N$

ปัญหาย่อย

ปัญหาย่อย	คะแนน	เงื่อนไขเพิ่มเติม
1	5	$T[i] = 1$ สำหรับแต่ละ i ที่ $0 \leq i < N$
2	7	$N \leq 3000$; $T[i] \leq 2$ สำหรับแต่ละ i ที่ $0 \leq i < N$
3	12	$T[i] \leq 2$ สำหรับแต่ละ i ที่ $0 \leq i < N$
4	15	$N \leq 70$
5	27	ไม่สามารถซื้อคูปองได้ครบทั้ง N คูปอง (ในสัปดาห์หนึ่ง)
6	16	$(A - P[i]) \cdot T[i] < A$ สำหรับแต่ละ i ที่ $0 \leq i < N$
7	18	ไม่มีเงื่อนไขเพิ่มเติม

ตัวอย่าง

ตัวอย่าง 1

พิจารณาการเรียกฟังก์ชันดังนี้

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

ในตอนเริ่มเกม ในเรามีเหรียญ $A = 13$ เหรียญ เธอสามารถซื้อคูปองได้ 3 ใบตามลำดับ ดังตารางด้านล่างนี้:

หมายเลขคูปองที่ซื้อ	ราคาของคูปอง	ประเภทของคูปอง	จำนวนเหรียญที่เหลืออยู่หลังการซื้อ
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

ในตัวอย่างนี้ ในเราไม่สามารถซื้อคูปองได้มากกว่า 3 ใบ และลำดับในการซื้อที่อธิบายด้านบนเป็นวิธีเดียวที่เธอจะซื้อคูปองได้ 3 ใบ ดังนั้นฟังก์ชันต้องคืนค่า $[2, 3, 0]$

ตัวอย่าง 2

พิจารณาการเรียกฟังก์ชันดังนี้

```
max_coupons(9, [6, 5], [2, 3])
```

ในตัวอย่างนี้ ในเราสามารถซื้อคูปองในลำดับใดก็ได้ ดังนั้นฟังก์ชันต้องคืนค่า $[0, 1]$ หรือ $[1, 0]$

ตัวอย่าง 3

พิจารณาการเรียกฟังก์ชันดังนี้

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

ในตัวอย่างนี้ ในเรามีเหรียญเพียงหนึ่งเหรียญซึ่งไม่เพียงพอที่จะซื้อคูปองใด ๆ ดังนั้นฟังก์ชันต้องคืนค่า $[]$ (อาร์เรย์ว่าง)

เกรตเตอร์ตัวอย่าง

รูปแบบข้อมูลนำเข้า:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

รูปแบบข้อมูลส่งออก:

```
S
R[0] R[1] ... R[S-1]
```

ในที่นี้ S เป็นความยาวของอาร์เรย์ R ที่ถูกคืนโดยฟังก์ชัน `max_coupons`